

Apache Spark Cluster Modes

Sure! Let's talk about the different cluster modes in Apache Spark.

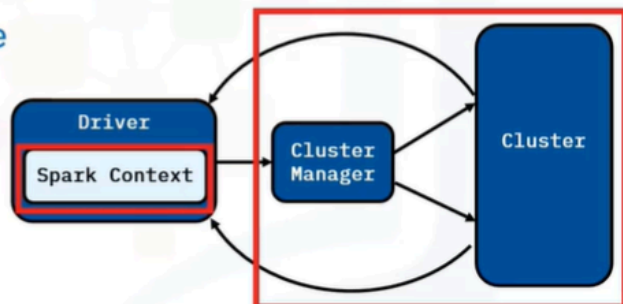
In simple terms, Apache Spark can run in various ways, depending on how you want to manage your resources. Think of it like choosing different modes of transportation for a journey. You can drive a car (Spark Standalone), take a bus (Apache Hadoop YARN), share a ride with friends (Apache Mesos), or hop on a train (Kubernetes). Each mode has its own benefits and is suited for different situations. For example, Spark Standalone is like a personal car—easy to set up and great for small trips. In contrast, YARN is more like a bus system that can handle many passengers but requires more planning.

When you want to run Spark, you need to tell it which mode to use. This is done by specifying the cluster manager type. For instance, if you want to use Spark Standalone, you would start by setting up a Master node, which acts like the driver, coordinating the workers (the passengers) to get the job done. Each mode has its own setup requirements, so it's essential to choose the one that fits your needs best.

Spark Cluster Manager

Spark Cluster Manager:

- Communicates with a particular cluster to acquire resources for the running application
- Runs as a service outside the application and abstracts the cluster type



- The Spark Cluster Manager communicates with a cluster to acquire resources for an application to run. It runs as a service outside the application and abstracts the cluster type. While an application is running, the Spark Context creates tasks and communicates to the cluster manager what resources are needed. Then the cluster manager reserves executor cores and memory resources. Once the resources are reserved, tasks can be transferred to the executor processes to run.

Types of Cluster Managers

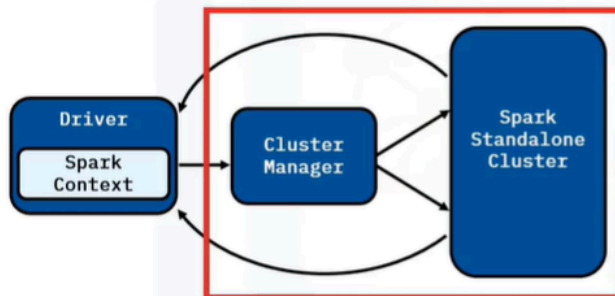
Spark supports the following cluster managers:

- **Spark Standalone** – comes with Spark, best for setting up a simple cluster
- **Apache Hadoop YARN** (Yet Another Resource Negotiator) – cluster manager from the Hadoop project
- **Apache Mesos** – general-purpose cluster manager with additional benefits
- **Kubernetes** – open-source system for running containerized applications



- Spark has built-in support for several cluster managers. A Standalone cluster manager is included with the Spark installation. It's best for setting up a simple cluster. Apache Hadoop YARN, or Yet Another Resource Negotiator, is a cluster manager from the Hadoop project. Apache Mesos is a general-purpose cluster manager that Spark can run with additional benefits. Finally, Kubernetes is an open-source system for running containerized applications.

Why use Spark Standalone



Spark Standalone benefits are:

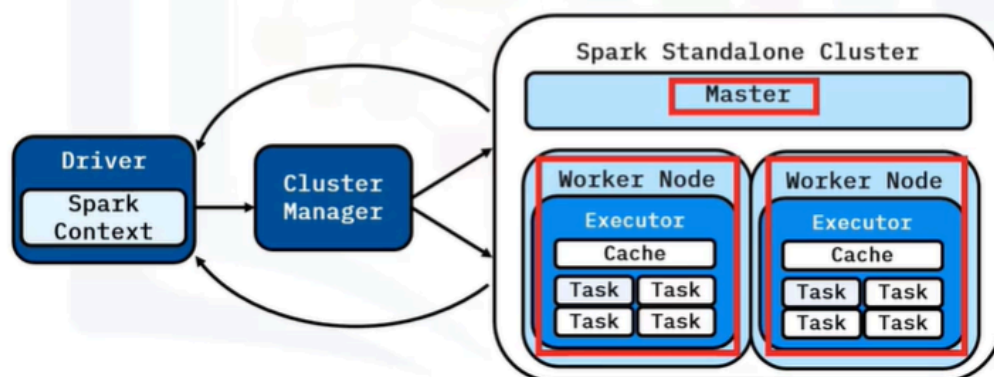
- Built into the Spark installation, so no additional dependencies to deploy
- Fastest way to setup a Spark cluster and get running
- Specifically designed for Spark, not "general-purpose"

- Spark Standalone comes packaged with the Spark installation, so there are no additional dependencies required to configure and deploy. Spark Standalone is specifically designed to run Spark and is often the fastest way to get a cluster up and running applications

Spark Standalone components

Spark Standalone has two main components:

- Workers - run an Executor process to receive tasks
- Master - connects and adds workers to the cluster



- A Spark Standalone cluster has two main components: Workers and the Master. The workers run on cluster nodes. They start an executor process with one or more reserved cores. There must be one master available which can run on any cluster node. It connects workers to the cluster and keeps track of them with heartbeat polling. However, if the master is together with a worker, do not reserve all the node's cores and memory for the worker.

How to set up Spark Standalone

1. Start the Master to output the Master URL

```
$ ./sbin/start-master.sh
```

2. Start Worker(s) with the Master URL

```
$ ./sbin/start-slave.sh spark://<master-spark-URL>:7077
```

3. Launch Spark application on cluster by specifying the Master URL

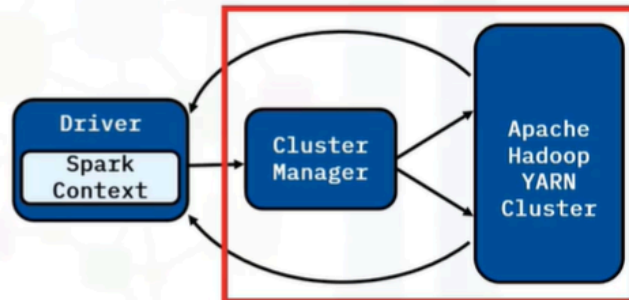
```
$ ./bin/spark-submit \  
  --master spark://<spark-master-URL>:7077 \  
  <additional configuration>
```

- To manually set up a Spark Standalone cluster, start the Master. The Master is assigned a URL with a hostname and port number. After the master is up, you can use the Master URL to start workers on any node using bi-directional communication with the master. Once the master and the workers are running, you can launch a Spark application on the cluster by specifying the master URL as an argument to connect them. To see additional configuration options, refer to the Apache Spark documentation on the spark.apache.org website. Apache Hadoop YARN is a general-purpose cluster manager.

Why use Apache Hadoop YARN

When choosing Apache Hadoop YARN, consider that it:

- Is general-purpose
- Supports many other big data ecosystem frameworks
- Requires its own configuration and setup
- Has dependencies, making it more complex to deploy than Spark Standalone



- It's popular in the big data ecosystem and supports many other frameworks besides Spark. YARN clusters have their own dependencies, set-up and configuration requirements so deploying them is more complex than Spark Standalone.

How to run Spark on existing YARN cluster

1. Specify the Master option '--master YARN' with 'spark-submit'

```
$ ./bin/spark-submit \  
  --master YARN \  
  <additional configuration>
```

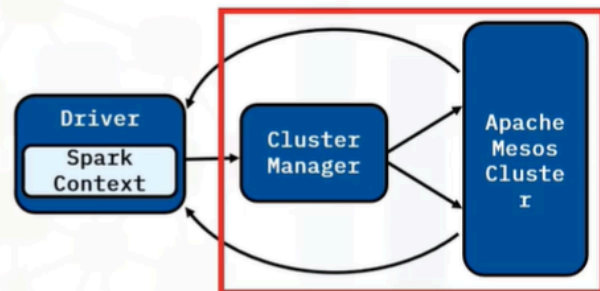
2. Spark will automatically connect with YARN using Hadoop configuration

- To run Spark on an existing YARN cluster, use the '--master' option with the keyword YARN. When Spark sees that the cluster manager type is YARN, it will look for the standard Hadoop configuration files to decide how to connect with the YARN cluster. No other configuration from Spark is needed.

Why use Apache Mesos

Apache Mesos Cluster Managers can run Spark with other benefits, such as making partitioning:

- Scalable between many Spark instances
- Dynamic between Spark and other big data frameworks



To configure Spark to run on an Apache Mesos cluster manager see:

<https://spark.apache.org/docs/latest/running-on-mesos.html>

- Apache Mesos is another general-purpose cluster manager supported by Spark. Using Mesos has some advantages. It can provide dynamic partitioning between Spark and other big data frameworks and scalable partitioning between multiple Spark instances. However, running Spark on Apache Mesos might require some additional setup depending on your configuration requirements. More details are provided on the spark.apache.org website at the link shown here.

Why use Spark on Kubernetes

Kubernetes cluster managers can run containerized applications, making it easier to:

- Automate deployment
- Simplify dependency management
- Scale the cluster

How to run Spark on Kubernetes

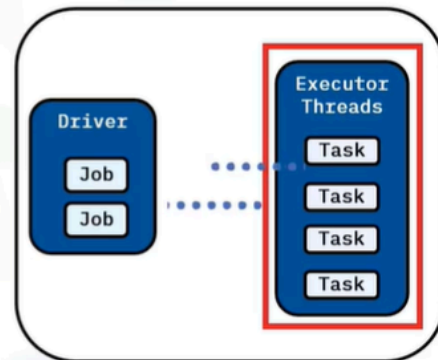
To launch a Spark Application on Kubernetes use:

```
$ ./bin/spark-submit \  
  --master k8s://https://<k8s-apiserver-host>:<k8s-apiserver-port> \  
  <additional configuration>
```

Why use Local Mode

Spark can also be run in local mode which:

- Does not connect to a cluster, making it easy to get started
- Runs in the same process that calls 'spark-submit' and uses threads for running executor tasks
- Can be useful for testing or debugging a Spark application
- Runs a Spark application locally within a single process which can limit performance



- Local mode runs a Spark application as a single process locally on the machine. Executors are run as separate threads in the main process that calls 'spark-submit'. Local mode does not connect to any cluster or require configuration outside a basic Spark installation. Local mode can be run on a laptop. That's useful for testing or debugging a Spark application, for example, testing a small data subset to verify correctness before running the application on a cluster. However, being constrained by a single process means local mode is not designed for optimal performance.

How to set up Local Mode

To run Spark in local mode, use the master option `--master local[#]` where `#` specifies the number of cores to use

```
# Launch Spark in local mode with 8 cores
$ ./bin/spark-submit \
  --master local[8] \
  <additional configuration>
```

Use an asterisk `*` to specify using all available cores

```
# Launch Spark in local mode with all available cores
$ ./bin/spark-submit \
  --master local[*] \
  <additional configuration>
```

- To run in local mode, use the `--master` option with the keyword `'local'` followed by a number to indicate how many cores the Spark application can use for the executor. To use all available cores, replace the number with an asterisk wildcard. Keep in mind not all configuration for a cluster will be valid in local mode.

Summary

In this video, you learned that:

- Cluster Managers acquire resources and run as an abstracted service outside the application
- Spark can run on Spark Standalone, Apache Hadoop YARN, Apache Mesos or Kubernetes Cluster Managers, with specific set-up requirements
- Selecting the right Cluster Manager depends on your data ecosystem and factors such as ease of configuration, portability, deployment or data partitioning needs
- Spark can run in local mode (using master option `--master local[#]`), which is helpful to test or debug an application before deployment

